MetaKG: A Unified Knowledge Graph System for Metabolic Pathway Analysis

Eric G. Suchanek, PhD[1]

[1]Flux Frontiers, github.com/Flux-Frontiers

February 2026

Abstract

We present MetaKG, a local-first metabolic knowledge graph whose dual-layer architecture unifies structural precision and semantic expressivity in a single system: SQLite encodes the full property graph of compounds, reactions, enzymes, and pathways with stoichiometric and regulatory relationships, while LanceDB maintains dense vector embeddings for natural-language semantic search over the same entities. This design delivers four orthogonal query modalities—natural-language pathway discovery, structural neighbourhood traversal, shortest-path search between metabolites, and stoichiometric detail assembly—without forcing the user to choose between relational precision and semantic exploration. A post-build name-enrichment pipeline replaces bare KEGG accessions with human-readable names sourced from KEGG bulk name lists, making all query results immediately interpretable. Beyond querying, MetaKG integrates three simulation modalities directly into the knowledge graph: Flux Balance Analysis for steady-state flux optimisation, kinetic ODE integration with Michaelis-Menten rate laws (seeded from BRENDA and SABIO-RK) using an implicit stiff solver (BDF), and what-if perturbation analysis for enzyme knockouts and substrate overrides. Interactive exploration is supported by a 2D Streamlit graph browser and a 3D PyVista visualiser with Fibonacci-annulus and layer-stratified layouts (both under active development). All capabilities are exposed through a unified Python API, command-line interface, and Model Context Protocol (MCP) server exposing nine typed tools, making MetaKG a first-class data source for AI reasoning systems. We demonstrate the system on the complete *Homo sapiens* metabolome—all 369 KEGG pathways ingested as a single queryable graph (17,050 nodes, 11,286 edges, 9,427 enzymes, 20,151 indexed embeddings)—with structural queries completing in 10–50 ms, semantic searches in 100–500 ms, and kinetic simulations in under 400 ms on commodity hardware.[1]

Keywords: metabolic pathways, knowledge graph, dual-layer architecture, semantic search, local-first systems, AI-accessible data, multi-format integration, bioinformatics

1 Introduction

1.1 The Problem: Query Architecture Limits Exploratory Analysis

Reconstructing metabolic networks means integrating data from diverse sources: KEGG provides human-curated pathway maps in KGML [Kanehisa et al., 2021]; Reactome and model-organism databases export in BioPAX Level 3 [Demir et al., 2010]; constraint-based modelling tools produce SBML [Keating et al., 2020]; and institutional data often lives only in spreadsheets. Handling multiple formats is a technical necessity, but it is not the core scientific problem. The deeper issue is *query capability*.

Existing systems force a choice between two incompatible paradigms:

Structural precision. Systems like KEGG, BioCyc, and Reactome support exact structural queries—"retrieve all products of this enzyme" or "find the shortest path between two compounds"—through relational

[1]All timings measured on a 2024 MacBook Pro (Apple M3 Max, 36 GB RAM, 1 TB SSD).

data models or graph databases. Yet they lack semantic search: a biologist looking for "fatty-acid oxidation" gets keyword matches, not conceptually similar pathways. They also require web access and resist programmatic querying at scale.

Semantic expressivity. Vector databases and embeddings handle semantic similarity well—"find pathways similar to X"—and cope with synonymy and domain terminology. But they sacrifice property graph structure; stoichiometric detail, cofactor roles, and regulatory logic disappear. Semantic-only systems are exploration tools, not precision instruments.

No existing system unites structural precision, semantic expressivity, and programmatic accessibility in a single local tool. KEGG and PathBank are web-only. MetaNetX reconciles identifiers but offers no queryable graph. Neo4j-based systems handle complex structural queries but require server deployment and do not address parsing. Vector databases excel at search but discard relational structure. Put simply, no platform lets an analyst start with a semantic query ("what pathways relate to glucose metabolism?"), drill into structural detail ("what is the precise stoichiometry of this reaction?"), and assemble the results—all within a single local, reproducible, programmable system.

1.2 The Solution: Dual-Layer Query Architecture

MetaKG resolves this tension with a dual-layer local knowledge graph that eliminates the forced choice between structural precision and semantic expressivity. The key idea is to separate query problems by modality:

Structural layer. SQLite stores the property graph (compounds, reactions, pathways, and their relationships). Structural queries run as efficient SQL joins: neighbourhood traversal, shortest-path BFS, stoichiometric assembly.

Semantic layer. LanceDB maintains a vector index over node descriptions using sentence-transformer embeddings. Semantic queries retrieve results by cosine similarity: natural-language pathway search, compound similarity, synonym resolution.

Together, these layers give analysts access to four query modalities on the same graph, without committing to a single paradigm. A researcher can begin with semantic exploration and then drill into structural detail, all in one interface:

1. **Semantic pathway discovery** — "Find pathways related to fatty-acid beta-oxidation" (vector similarity search; handles synonyms, abbreviations, and domain terminology)
2. **Structural neighbourhood traversal** — "Find all compounds that are products of pyruvate carboxylase" (SQL joins)
3. **Shortest-path search** — "What is the minimal metabolic route from glucose to acetyl-CoA?" (BFS over the compound–reaction bipartite graph)
4. **Stoichiometric detail assembly** — "Retrieve all substrates, products, enzymes, inhibitors, and cofactors for reaction R00200" (multi-table JOIN with JSON unpacking)

Multi-format parsing (KGML, SBML, BioPAX, CSV) is the enabling infrastructure. It lets users ingest data from any source and merge it through stable, deterministic identifiers. But parsing is table stakes. The innovation is the dual-layer query engine: it frees researchers from choosing between semantic expressivity and structural precision.

All four query modalities are exposed through a unified Python API, command-line interface, and Model Context Protocol (MCP) server. The system runs entirely locally—no network, no database server, no external services after setup. This is a deliberate design choice. Unlike live database mirrors, MetaKG treats the knowledge graph as a reproducible, version-controlled snapshot. Users rebuild when input files change, enabling reproducible analysis, offline workflows, and integration into research pipelines. The system scales to organism-scale pathway corpora: the complete *Homo sapiens* metabolome (369 KEGG pathways, 17,050 nodes, 11,286 edges, 9,427 enzymes) builds in 30–60 seconds, and queries complete in milliseconds to seconds.

1.3 Design Goals

MetaKG is built around three core principles, in priority order:

1. **Dual-layer query architecture.** Combine structural and semantic queries in a single system. Do not force users to choose between precise graph traversal and exploratory semantic search; both modalities should work seamlessly on the same data through the same API.
2. **Format-agnostic, deterministic merging.** Accept pathway data in any of four formats (KGML, SBML, BioPAX, CSV) and produce a unified graph with stable, deterministic identifiers. Enable reproducible builds and incremental updates without external reconciliation services.
3. **Zero-friction local deployment.** Require no database servers, external services, or network connections after setup. The entire stack—parsing, storage, indexing, querying, visualisation, MCP server—fits in a single Python library, runnable on a laptop or in batch workflows.

The remainder of this paper is organised as follows. Section 2 describes the data model and system architecture. Section 3 covers the format-specific parsers. Section 4 describes the name-enrichment pipeline. Section 5 details the storage and indexing layers. Section 6 describes the query API. Section 7 covers the visualisation components. Section 8 describes the MCP server. Section 9 walks through a complete worked example. Section 11 discusses limitations and future directions, and Section 12 concludes.

2 Data Model and Architecture

2.1 Property Graph Schema

MetaKG represents metabolism as a directed property graph $G = (V, E)$. Vertices V are *entities* of four kinds:

`compound` A small-molecule metabolite. May carry a molecular formula, net formal charge, and cross-references to external databases (ChEBI, HMDB, PubChem, InChI).

`reaction` A biochemical transformation. Carries stoichiometry encoded as a JSON object listing substrates and products with their coefficients, a reversibility flag, and cross-references to KEGG and Rhea.

`enzyme` A protein catalyst. Carries an EC number and cross-references to UniProt and NCBI Gene.

`pathway` An ordered or thematic collection of reactions, typically corresponding to one named pathway in a source database.

Edges E carry one of seven relation types (Table 1). All edges are directed and may include an evidence blob encoded as JSON, which parsers use to record stoichiometric coefficients, compartment labels, and source-specific annotations.

Table 1: Edge relation types in the MetaKG graph schema.

Relation	Source kind	Target kind
`SUBSTRATE_OF`	compound	reaction
`PRODUCT_OF`	reaction	compound
`CATALYZES`	enzyme	reaction
`INHIBITS`	compound	reaction
`ACTIVATES`	compound	reaction
`CONTAINS`	pathway	reaction
`XREF`	any	any

2.2 Stable Identifier Scheme

Identifier reconciliation is one of the persistent difficulties in biological data integration. MetaKG assigns each node a stable, URI-style string identifier of the form:

```
<prefix>:<namespace>:<external-id>
```

where the prefix encodes the node kind (`cpd`, `rxn`, `enz`, `pwy`) and the namespace identifies the source database (`kegg`, `chebi`, `uniprot`, `ec`, etc.):

```
cpd:kegg:C00022       # Pyruvate (KEGG)
rxn:kegg:R00200       # Pyruvate decarboxylation
enz:ec:1.2.4.1        # Pyruvate dehydrogenase (EC)
pwy:kegg:hsa00010     # Glycolysis / Gluconeogenesis
```

Listing 1: Examples of stable node identifiers.

When an entity appears in a file without an external database identifier, the system constructs a synthetic identifier by hashing the lowercased display name with SHA-1 and retaining the first eight hexadecimal digits:

```
cpd:syn:a4f2b8c1
```

Because the hash operates on the normalised name, identifiers are deterministic across independent parser runs on the same input. This makes incremental rebuilds possible without producing duplicate nodes. When two source files refer to the same KEGG or ChEBI entry, the graph merges their nodes by ID before writing to SQLite, so each logical entity appears exactly once regardless of how many files contributed to it.

2.3 System Architecture and the Dual-Layer Query Engine

Figure 1 shows the overall pipeline. The `MetaKG` orchestrator class owns the full sequence from raw files to query results, coordinating three subsystems:

1. **MetabolicGraph** — handles file discovery and parser dispatch. Outputs normalised `MetaNode` and `MetaEdge` objects that are independent of source format.
2. **MetaStore** — the SQLite persistence layer for structural graph queries (neighbourhood traversal, shortest-path BFS, stoichiometric assembly) via efficient SQL joins. Provides ACID guarantees with no external dependencies.
3. **MetaIndex** — the LanceDB vector index layer for semantic queries (natural-language pathway search, compound similarity retrieval). Uses pre-trained sentence-transformer embeddings to handle synonymy, abbreviations, and domain terminology.

The dual-layer design is the core architectural contribution. By routing structural queries (naturally SQL problems) to SQLite and semantic queries (naturally vector problems) to LanceDB, MetaKG avoids forcing a choice between relational precision and semantic expressivity. A user can traverse the graph by stoichiometry using SQL, then search for semantically similar pathways using embeddings—all within the same interface.

Both the store and the index initialise lazily: a caller that builds only the SQLite database (e.g. with `-no-index`) never loads the embedding model. This supports lightweight deployments where only structural queries are needed, avoiding the 100 MB embedding model download unless semantic search is actually used.

Listing 2: High-level data flow.

```
Pathway files (KGML / SBML / BioPAX / CSV)
        |
  MetabolicGraph
  (file discovery + parser dispatch)
        |
  MetaNode / MetaEdge objects
  (merged by stable ID)
        |
  MetaStore  ----> SQLite
  (write + xref index)
        |
  MetaIndex  ----> LanceDB
  (sentence-transformer embeddings)
        |
  Name Enricher (optional post-build pass)
  Phase 1: enzyme labels from CATALYZES edges
  Phase 2: KEGG TSV -> human-readable names
        |
  Query API + CLI + MCP server
```

Figure 1: MetaKG data pipeline. Format-specific parsers produce a stream of normalised `MetaNode` and `MetaEdge` objects that are merged by stable identifier, persisted to SQLite, and indexed as dense vectors in LanceDB. An optional name-enrichment pass replaces bare KEGG accessions with human-readable names.

3 Format-Specific Parsers

MetaKG ingests metabolic pathway data from four standard formats: KEGG Markup Language (KGML), Systems Biology Markup Language (SBML), Biological Pathway Exchange (BioPAX), and plain tabular files (CSV/TSV). All parsers conform to an abstract interface: they detect the file type, parse it, and produce normalised `MetaNode` and `MetaEdge` objects. Parsing is stateless and deterministic—the same input file always yields the same output.

Format detection relies on content analysis (the root XML element) rather than file extension, making the system robust to files served without standard extensions. The parser dispatcher examines each file, selects the appropriate handler, and produces a unified stream of nodes and edges. These are then merged by stable identifier in the MetaStore layer. Detailed parser specifications for each format appear in Appendix A.

4 Name Enrichment

KGML-sourced graphs initially carry bare KEGG accessions as node names (e.g. `C00031`, `R00710`). While these identifiers are unambiguous, they are not human-readable and impede exploratory analysis. MetaKG provides an optional post-build enrichment pipeline that replaces accessions with canonical names and stores them directly in `meta_nodes.name`, making them available everywhere—CLI, MCP, Streamlit, and the Python API.

4.1 Two-Phase Enrichment

Enrichment proceeds in two phases, both idempotent and safe to re-run:

Phase 1 (no network, always runs). For every reaction node still carrying a bare accession name, the enricher looks up all `CATALYZES` edges pointing to it and joins the catalysing enzyme gene symbols with a slash separator:

$$\texttt{R00710} \rightarrow \texttt{ADH1A / ADH1B / ADH1C}$$

This phase requires no external data and runs immediately after build.

Phase 2 (requires downloaded KEGG name lists). KEGG publishes bulk name lists for compounds (∼19,500 entries) and reactions (∼12,400 entries). A provided download script fetches these files once; subsequent enrichment runs read them locally:

```
python scripts/download_kegg_names.py
metakg-enrich --db .metakg/meta.sqlite
```

Phase 2 updates compound names (e.g. `C00031` → `D-Glucose`) and reaction names (e.g. `R00710` → `Acetaldehyde:NAD⁺ oxidoreductase`), overriding Phase 1 enzyme labels where canonical KEGG names are available.

Both phases can be combined with the build step via the `-enrich` flag:

```
metakg-build --data ./pathways --enrich
```

After enrichment, all query results—including MCP tool responses and Streamlit displays—show human-readable names rather than bare accessions. The enrichment is stored in the database, so it persists across sessions without re-running.

5 Storage and Indexing

The dual-layer storage architecture is what gives MetaKG its query flexibility. Rather than routing all queries through a single backend (as relational databases do) or abandoning structure entirely (as vector-only systems do), it maintains two complementary stores, each optimised for a different class of query:

SQLite handles efficient relational queries over graph structure. Compound–reaction–pathway relationships map naturally to SQL joins. Shortest-path searches use BFS over edges. Stoichiometric detail (substrates, products, coefficients) is stored as decoded JSON columns.

LanceDB + embeddings handles semantic search over node descriptions. Users query in natural language ("fatty-acid oxidation"), and the system returns semantically similar pathways via vector similarity. This matters most for exploratory analysis where the user does not know exact identifiers.

The two layers are not redundant—they solve fundamentally different problems. A researcher might start with semantic discovery ("which pathways involve energy metabolism?"), then drill into structural detail ("what is the precise stoichiometry of ATP synthesis?"), all without leaving the interface.

5.1 SQLite Layer

Parsed nodes and edges are written to a SQLite database through the `MetaStore` class. The schema uses three tables:

`meta_nodes` One row per node. Columns correspond to the fields of `MetaNode`: `id`, `kind`, `name`, `description`, `formula`, `charge`, `ec_number`, `stoichiometry` (JSON), `xrefs` (JSON), `source_format`, `source_file`. Indexed on `kind` and `name`.

`meta_edges` One row per directed edge: `src`, `rel`, `dst`, `evidence` (JSON). Indexed on `src`, `dst`, and `rel`.

`xref_index` A materialised inverse mapping from each external identifier to the corresponding internal node ID, built after all nodes have been written. This enables look-up by KEGG compound ID, ChEBI accession, UniProt accession, or any other cross-reference stored in the `xrefs` JSON blob.

SQLite runs with write-ahead logging (`WAL` journal mode) and `NORMAL` synchronisation; these pragmas deliver throughput close to an in-memory database while preserving crash safety for workloads that write once and read many times.

5.2 Semantic Index

Structural queries alone are insufficient for exploratory use cases where the user does not know an exact identifier. MetaKG therefore maintains a vector index over node descriptions using LanceDB [LanceDB Contributors, 2023] as the approximate nearest-neighbour engine and the `all-MiniLM-L6-v2` sentence-transformer model [Reimers and Gurevych, 2019] as the default encoder. This model produces 384-dimensional embeddings and occupies approximately 100 MB of disk space on first download.

Each node is serialised to an embedding string that concatenates its name, molecular formula (for compounds), EC number (for enzymes), cross-reference values, and free-text description. Reactions are excluded from the vector index because reaction identity is better captured by stoichiometric connectivity, which is available through the SQLite layer. Compounds, enzymes, and pathways are indexed.

The `MetaIndex.search(query, k)` method embeds the query string with the same model and returns the `k` approximate nearest neighbours, along with their cosine distances. Results are then joined against SQLite to return full node metadata.

Users may substitute any encoder that implements the `Embedder` abstract class, which requires only a single method:

```
class Embedder(Protocol):
    def encode(self,
               texts: list[str]
               ) -> list[list[float]]: ...
```

6 Query API

MetaKG exposes four query modalities through a unified API. These operations share the same underlying graph but use different access patterns suited to different analysis tasks. Together, they support both precision (exact structural queries) and exploration (semantic discovery).

6.1 Node Retrieval and Neighbourhood Traversal

`MetaStore.node(id)` fetches a single node by internal ID. The returned dictionary mirrors the `meta_nodes` schema, with the `xrefs` and `stoichiometry` fields pre-decoded from JSON.

`MetaStore.neighbours(id, rels=...)` returns all nodes reachable from the given node by following the specified relation types. The default relation tuple is `(SUBSTRATE_OF, PRODUCT_OF, CATALYZES, CONTAINS)`. A single SQL query over the `meta_edges` table resolves both outgoing and incoming edges and joins the results against `meta_nodes`.

6.2 Reaction Detail

`MetaStore.reaction_detail(id)` assembles a structured view of a single reaction, returning a dictionary with keys `substrates`, `products`, `enzymes`, `inhibitors`, `activators`, and `pathways`. Each value is a list of node dictionaries. This serves as the primary access point for stoichiometric analysis.

6.3 Shortest-Path Search

`MetaStore.find_shortest_path(a, b, max_hops)` performs an iterative breadth-first search over the SQLite graph, alternating between `SUBSTRATE_OF` and `PRODUCT_OF` edges to traverse the bipartite compound–reaction graph. The search terminates when it reaches the target node or exceeds the hop limit. Both internal IDs and external identifiers (resolved through `xref_index`) are accepted as arguments.

The algorithm runs entirely in Python with SQL queries per BFS frontier, which is practical for graphs typical of a single organism's curated metabolic network (tens of thousands of nodes). For corpora with hundreds of thousands of reactions, a specialised graph engine would be more appropriate.

6.4 Semantic Search

`MetaKG.query_pathway(name, k)` performs a vector search over the LanceDB index, filtering results to the `pathway` node kind. The raw `MetaIndex.search(query, k)` method returns hits from all indexed kinds. Both methods accept free-text queries in natural language; the sentence-transformer model handles biological terminology effectively because its training corpus includes scientific text.

`MetaKG.resolve_id(s)` provides unified look-up that accepts any of: an internal ID (`cpd:kegg:C00022`), a shorthand external reference (`kegg:C00022`), or a display name (`Pyruvate`). It queries `xref_index` first for exact matches, then falls back to a case-insensitive name search in `meta_nodes`. This lets the same user-facing functions accept identifiers from any source database.

6.5 Metabolic Simulations

Building on the structural query layer, MetaKG provides three simulation modalities:

1. **Flux Balance Analysis (FBA)** — `MetaKG.simulate_fba(pathway_id, maximize=True)` performs steady-state optimisation using the stoichiometry stored in the graph. The result includes a status flag, objective value, and per-reaction flux distribution.
2. **Kinetic ODE Integration** — `MetaKG.simulate_ode(pathway_id, t_end, t_points, ...)` runs time-course simulation with Michaelis-Menten rate laws and seeded kinetic parameters (Km, Vmax, kcat). It uses an implicit stiff solver (BDF) tuned for metabolic systems and returns time-course concentration trajectories for all compounds.
3. **What-If Perturbation Analysis** — `MetaKG.simulate_whatif(pathway_id, scenario_json, mode)` compares a baseline pathway against a perturbed version with enzyme knockouts, inhibition, or substrate concentration overrides. Both FBA and ODE modes are supported.

Kinetic parameters are populated on first use by seeding from literature sources (BRENDA, SABIO-RK, published metabolic models) via `MetaKG.seed_kinetics()`.

7 Visualisation

7.1 2D Web Explorer

The `metakg-viz` command launches a Streamlit [Streamlit Inc., 2019] web application with three views:

1. **Graph Browser** — an interactive network rendered with pyvis [West, 2021] and displayed in the browser. Nodes are colour-coded by kind (pathway: blue, reaction: red, compound: green, enzyme: orange) and edges by relation type. A sidebar provides filters by node kind and limits the number of rendered nodes to keep the visualisation manageable for large graphs.
2. **Semantic Search** — a free-text query box that calls `MetaIndex.search` and displays ranked results with similarity scores.
3. **Node Details** — clicking any node opens a detail panel showing all metadata and the node's immediate neighbourhood.

7.2 3D Visualiser

The `metakg-viz3d` command launches a PyVista [Sullivan and Kaszynski, 2019] interactive 3D viewer. Two layout strategies are available:

Allium layout. Each pathway node sits on a flat Fibonacci annulus in the XY-plane [Vogel, 1979]. Reaction and compound nodes belonging to a pathway are distributed on a Fibonacci sphere centred on the pathway's position, creating a visual metaphor of an inflorescence. Nodes belonging to multiple pathways are placed at the centroid of their pathway positions, so cross-pathway metabolites appear in intermediate locations.

LayerCake layout. Nodes are stratified by kind along the Z-axis: pathways occupy the lowest layer, reactions the middle, and compounds and enzymes the top. Within each layer, nodes follow a golden-angle spiral to minimise overlap. This layout suits inspection of the bipartite compound–reaction structure.

Both layouts export to HTML (for web reports) and PNG (for publication figures).

8 Model Context Protocol Server

The Model Context Protocol (MCP) [Anthropic, 2024b] is a lightweight JSON-RPC standard that lets large-language-model assistants call typed tool functions. MetaKG implements an MCP server exposing the knowledge graph through nine typed tools:

`query_pathway(name, k)` Semantic pathway search. Returns up to `k` pathway nodes whose descriptions are closest to the query in embedding space, along with their member-reaction counts.

`get_compound(id)` Returns a compound node with its connected reactions, accepting any supported identifier format (full URI, shorthand, or display name).

`get_reaction(id)` Returns full stoichiometric detail for a single reaction, including substrates, products, and catalysing enzymes.

`find_path(a, b, max_hops)` Returns the shortest metabolic path between two compounds via bidirectional BFS.

`simulate_fba(pathway_id, ...)` Runs Flux Balance Analysis on a pathway and returns the optimal flux distribution and shadow prices.

`simulate_ode(pathway_id, ...)` Runs kinetic ODE simulation (BDF solver) and returns time-course concentration trajectories.

`simulate_whatif(pathway_id, scenario_json, mode)` Compares baseline and perturbed simulations; supports enzyme knockouts, activity scaling, and substrate overrides.

`get_kinetic_params(reaction_id)` Retrieves stored kinetic and regulatory parameters (Km, Vmax, kcat, Ki, $\Delta G^{\circ\prime}$) for a reaction.

`seed_kinetics(force)` Populates the database with curated literature kinetic parameters from BRENDA, SABIO-RK, and published models.

The server starts with:

```
metakg-mcp --db .metakg/meta.sqlite \
           --transport stdio
```

and communicates over standard input/output (the `stdio` transport) or as an HTTP server-sent events stream (the `sse` transport). The `stdio` transport is the standard configuration for Claude [Anthropic, 2024a] and compatible MCP clients.

9 Worked Example: Complete Human Metabolome

We demonstrate MetaKG on the complete *Homo sapiens* metabolome: all 369 KEGG pathways (metabolic, signaling, and regulatory). This corpus spans central carbon metabolism (glycolysis/gluconeogenesis, TCA cycle, pentose phosphate pathway, fatty acid degradation, oxidative phosphorylation), amino acid and nucleotide metabolism, secondary metabolism, carbohydrate metabolism, lipid metabolism, and signaling networks.

9.1 Building the Knowledge Graph

Listing 3: Building the complete human metabolome knowledge graph.

```
$ metakg-build --data ./data/hsa_pathways --wipe
Building MetaKG from ./data/hsa_pathways...
data_root   : ./data/hsa_pathways
db_path     : .metakg/meta.sqlite
nodes       : 22290  {'compound': 5115, 'enzyme': 14667,
              'pathway': 369, 'reaction': 2139}
edges       : 11298  {'CATALYZES': 2406, 'CONTAINS': 3809,
              'PRODUCT_OF': 2532, 'SUBSTRATE_OF': 2551}
indexed     : 20151 vectors  dim=384
```

The `-wipe` flag clears any prior database before parsing; omitting it allows incremental addition of new pathway files to an existing graph.

9.2 Structural Queries via the Python API

```python
from metakg import MetaKG

kg = MetaKG()

# Retrieve pyruvate and its connected reactions
cpd = kg.get_compound("cpd:kegg:C00022")
print(cpd["name"])
for rxn in cpd["reactions"]:
    print(f"  {rxn['name']:30s} {rxn['role']}")

# Full reaction detail
rxn = kg.get_reaction("rxn:kegg:R00200")
print(f"Substrates: {[s['name'] for s in rxn['substrates']]}")
print(f"Products:   {[p['name'] for p in rxn['products']]}")
```

Listing 4: Compound retrieval and neighbourhood traversal.

Output:

```
Pyruvate
  R00703                          SUBSTRATE_OF
  R00014                          SUBSTRATE_OF
  R00431                          SUBSTRATE_OF
  R00209                          SUBSTRATE_OF
  R00200                          PRODUCT_OF
  ... and 5 more
Substrates: ['Phosphoenolpyruvate', 'ADP']
Products:   ['Pyruvate', 'ATP']
```

9.3 Shortest-Path Search

```python
from metakg import MetaKG

kg = MetaKG()

result = kg.find_path(
    "cpd:kegg:C00031",   # D-Glucose
    "cpd:kegg:C00022",   # Pyruvate
    max_hops=12,
)
print(f"Path length: {result['hops']} steps")
for node in result["path"]:
    print(f"  {node['kind']:10s} {node['name']}")
```

Listing 5: Finding the shortest metabolic route between two compounds.

Output:

```
Path length: 9 steps
  compound   C00031
  reaction   R00299
  compound   C00092
  reaction   R00771
  compound   C00085
  reaction   R00756
  compound   C00354
  reaction   R01068
  compound   C00118
  reaction   R01061
  compound   C00236
  reaction   R01662
  compound   C01159
  reaction   R09532
  compound   C00631
  reaction   R00658
  compound   C00074
  reaction   R00200
  compound   C00022
```

The query resolves in milliseconds on the local SQLite index. The algorithm scales efficiently to the complete human metabolome (17,050 nodes, 11,286 edges) using bidirectional BFS with early termination. Typical shortest-path queries complete in 10–50 ms.

9.4 Semantic Search

```python
from metakg import MetaKG

kg = MetaKG()

result = kg.query_pathway("fatty acid beta-oxidation", k=5)
for hit in result.hits:
    print(f"{hit['name']:40s}  dist={hit['_distance']:.3f}")
```

Listing 6: Semantic pathway retrieval.

Output:

```
Fatty acid degradation                    dist=1.174
alpha-Linolenic acid metabolism           dist=1.183
Fatty acid metabolism                     dist=1.200
Biosynthesis of unsaturated fatty acids   dist=1.245
Linoleic acid metabolism                  dist=1.252
```

The semantic search correctly identifies related pathways despite differences in nomenclature between the query and the KEGG pathway names. The sentence-transformer model handles synonymy and domain terminology effectively.

9.5 Pathway Analysis Report

The `metakg-analyze` command runs a seven-phase analysis and produces a structured Markdown report:

```
$ metakg-analyze --output analysis.md --top 10
```

The report covers: (1) aggregate graph statistics; (2) hub metabolites ranked by degree; (3) reactions ranked by stoichiometric complexity; (4) cross-pathway hub detection; (5) pairwise pathway coupling by shared metabolites; (6) topological features (dead-end compounds, isolated nodes); and (7) enzymes ranked by reaction count. On the complete human metabolome (369 pathways), ATP, NAD^+, coenzyme A, and pyruvate emerge as the top hub metabolites by degree—consistent with their known roles as central energy and carbon carriers. The analysis also reveals cross-pathway metabolite connectivity and identifies the reactions with highest stoichiometric complexity.

9.6 Mixed-Format Ingestion

To illustrate multi-format ingestion, one can combine KGML files with a BioPAX export from Reactome and a custom CSV table in a single data directory:

```
$ ls pathways/
hsa00010.xml    # KGML (KEGG)
R-HSA-70171.owl # BioPAX (Reactome)
custom_rxns.csv # CSV (in-house data)

$ metakg-build --data ./pathways --wipe
```

The parser dispatcher examines the root XML element of each `.xml` or `.owl` file and selects the appropriate parser; `.csv` and `.tsv` files go to the tabular parser. Nodes sharing a KEGG or ChEBI cross-reference across files are automatically merged in SQLite through the xref index.

10 Implementation Notes

Dependencies. The core package requires Python 3.12, `lancedb` $\geq$ `0.29`, `numpy` $\geq$ `1.24`, and `sentence-transformers` $\geq$ `2.7`. No network connection is needed at run time once the embedding model has been downloaded. Simulation support (FBA and ODE) requires the optional `scipy` $\geq$ `1.11` package installed via `-extras simulate`. BioPAX support requires the optional `rdflib` package; the 2D and 3D visualisers need optional extras installed via `poetry install -extras viz` or `-extras viz3d`.

Installation.

```
git clone https://github.com/Flux-Frontiers/meta_kg
cd meta_kg
poetry install --extras all
```

Thread safety. The SQLite connection uses WAL mode with a single Python object per process. The current implementation is not thread-safe; callers should create one `MetaKG` instance per process or protect a shared instance with a lock.

Incremental rebuild. The stable ID scheme makes incremental builds straightforward. Running `metakg-build` without `-wipe` issues `INSERT OR REPLACE` statements, updating existing nodes and appending new ones without duplicating entries.

Codebase self-analysis. MetaKG is itself analysed with its sister tool CodeKG [Flux Frontiers Contributors, 2025], which builds a structural and semantic knowledge graph of the Python source code. This enables navigating the MetaKG implementation via natural-language queries and confirms that the two tools share compatible architectural patterns. On the MetaKG codebase, the CodeKG static analysis produces 3,136 nodes and 2,920 edges spanning 27 modules, embedded into a 384-dimensional vector index of 290 vectors.

11 Discussion

11.1 Architectural Design Choices and Novel Aspects

Several deliberate design trade-offs distinguish MetaKG from existing systems. First, the dual-layer architecture (SQLite for structure, LanceDB for semantics) is novel in the metabolic pathway domain. Graph databases like Neo4j handle complex queries but carry operational overhead (server management, scaling, deployment) and do not address multi-format parsing. Specialised vector databases like Weaviate or Pinecone excel at semantic search but are poor fits for structural graph traversal and require network access. By pairing lightweight SQLite with local vector search, MetaKG delivers both capabilities in a self-contained package suited to exploratory research and reproducible analysis.

Second, the deterministic identifier scheme with synthetic hashing enables reproducible cross-format merging without a centralised reconciliation service. Unlike MetaNetX (which requires API calls to reconcile identifiers), MetaKG produces self-contained graphs. The same input files always yield the same output

graph, making the knowledge graph a version-controlled artefact that supports reproducible science and offline workflows.

Third, the four-modality query interface (structural, pathfinding, semantic, stoichiometric) is intentionally broad. An analyst can start with a natural-language semantic query ("fatty-acid beta-oxidation"), then drill into structural detail (shortest paths, stoichiometric coefficients) without switching tools. This contrasts with systems that specialise in a single query paradigm.

Fourth, the Model Context Protocol integration is forward-looking. As large-language-model assistants become standard tools in computational biology, making the knowledge graph a first-class data source for Claude, ChatGPT, and future assistants is a natural step. The MCP interface represents a design commitment to AI-accessible knowledge graphs, not merely another API layer.

11.2 Scope and Design Trade-offs

Snapshot-based operation. MetaKG is a local analysis tool, not a live database mirror. The knowledge graph is a snapshot at build time; users must re-run `metakg-build` after updating source files. This keeps the system self-contained and free of dependencies on external services during analysis. For research workflows where reproducibility matters most, this is an advantage. For applications requiring real-time data updates, a live database backend would be more suitable.

Scale. SQLite and in-process BFS handle graphs of up to roughly 100,000 nodes, covering full reconstructed metabolic networks for a single organism. For pan-genome or multi-species analyses—where node counts reach into the millions—a dedicated graph database engine (e.g., Neo4j [Robinson et al., 2015] or Kùzu [Feng et al., 2023]) and a distributed vector index would be preferable. The storage layer is designed to be replaceable: `MetaStore` and `MetaIndex` are concrete classes behind well-defined interfaces, so alternative backends can be substituted without changing the parser or query layers.

Identifier reconciliation. The current cross-reference merge relies on exact match of external identifiers. Two compounds sharing a biological identity but differing in stereochemistry or protonation state will not merge automatically. More thorough reconciliation would require integration with a name normalisation service such as MetaNetX [Moretti et al., 2021] or UMLS [Bodenreider, 2004].

Stoichiometric models and simulations. MetaKG provides three simulation modalities: (1) Flux Balance Analysis (FBA) for steady-state flux optimisation; (2) kinetic ODE integration with Michaelis-Menten rate laws using an implicit stiff solver (BDF) tuned for metabolic pathways; and (3) what-if perturbation analysis for enzyme knockouts, inhibition, and substrate overrides. Kinetic parameters are seeded from literature sources (BRENDA, SABIO-RK, published models). For advanced constraint-based modelling, the SBML parser preserves all information needed to reconstruct a COBRApy model.

Performance on complete human metabolome. The complete *Homo sapiens* metabolome (369 KEGG pathways, 17,050 nodes, 11,286 edges, 20,151 vector embeddings) shows the following performance characteristics:

Table 2: Performance on the complete human metabolome (17K nodes, 11K edges). All timings measured on a 2024 MacBook Pro (Apple M3 Max, 36 GB RAM, 1 TB SSD).

Operation	Time	Details
Build graph (parse + index)	30–60s	All 369 pathways, LanceDB + SQLite
Semantic search (natural language)	100–500ms	Vector similarity on 20K nodes
Shortest-path (6 hops)	10–50ms	BFS on 11K edges
ODE simulation (10 units)	150–400ms	BDF solver, 24 compounds
Streamlit rerun	0.5–1.5s	Batch query + session cache

All operations complete within practical timeframes for interactive exploration. The graph is well suited to research workflows demanding reproducible analysis, offline access, and programmatic querying.

Future directions. Planned extensions include graph-theoretic centrality measures (betweenness, closeness, eigenvector centrality) via NetworkX for hub ranking; a GraphQL query endpoint; integration with the UniProt and ChEBI REST APIs for on-demand annotation; differential pathway analysis across organisms; GPU-accelerated embedding for large datasets; and export to Cytoscape [Shannon et al., 2003] JSON for

interoperability with the broader network biology ecosystem. Advanced kinetic parameter optimisation (fitting from experimental data) is also planned.

12 Conclusion

MetaKG fills a gap in metabolic data integration. Existing systems force a choice among convenient web interfaces (limited programmability, no semantic search, web-dependent), specialised graph databases (high operational complexity, parsing unsolved), and reconciliation services (no queryable graph provided). MetaKG breaks this pattern with a dual-layer local knowledge graph that unifies multi-format pathway data and delivers four orthogonal query modalities—structural, pathfinding, semantic, and stoichiometric—through a single API, CLI, and LLM-accessible MCP interface.

The core contributions are: (1) a stable, deterministic identifier scheme enabling reproducible cross-format merging without external services; (2) a dual-layer storage architecture (SQLite + LanceDB) that eliminates the forced choice between relational precision and semantic expressivity; (3) a two-phase name-enrichment pipeline that replaces bare KEGG accessions with human-readable names, making all query results immediately interpretable without network access; (4) four unified query modalities accessible to analysts at all levels of programming expertise; and (5) a forward-looking MCP server exposing nine typed tools that make metabolic knowledge graphs first-class data sources for AI assistants.

The design philosophy—local-first, self-contained, snapshot-based—is deliberate and represents a clean separation from live database mirrors. This makes MetaKG especially well suited to research workflows where reproducibility, offline analysis, and version control matter most. For applications needing real-time data, larger graph corpora, or distributed deployment, the modular architecture allows the storage and index backends to be swapped.

We expect MetaKG to be immediately useful as: (1) a foundation for pathway analysis scripts; (2) a data preparation stage for machine-learning workflows; (3) an AI-accessible knowledge source for metabolic reasoning in large-language-model applications; and (4) a template for similar knowledge graph systems in related biological domains (protein interactions, gene regulatory networks, drug–target interactions).

The software is freely available at https://github.com/Flux-Frontiers/meta_kg under the Elastic License 2.0.

A Format-Specific Parsers

All parsers conform to an abstract base class:

```
class PathwayParser:
    def can_handle(self, path: Path) -> bool: ...
    def parse(self, path: Path
              ) -> tuple[list[MetaNode],
                         list[MetaEdge]]: ...
```

Parsers are stateless and pure: the same input file always produces the same output. The `MetabolicGraph` layer caches the combined node and edge lists after the first parse, so repeated calls do not re-read disk.

A.1 KGML Parser

KEGG Markup Language files are the native export format of the KEGG pathway database [Kanehisa et al., 2021]. Each file is an XML document whose root element is `<pathway>`. The parser uses the Python standard-library `ElementTree` module (no third-party XML dependency) and extracts three kinds of child elements:

- `<entry>` elements with `type="compound"` become `compound` nodes.
- `<entry>` elements with `type="gene"` or `type="enzyme"` become `enzyme` nodes.
- `<reaction>` elements become `reaction` nodes with their `<substrate>` and `<product>` children encoded as stoichiometry JSON. The enclosing pathway becomes a `CONTAINS` edge to each reaction.

Format detection checks the root element tag rather than the file extension, making the parser robust to KEGG files served without the `.kgml` extension.

A.2 SBML Parser

The Systems Biology Markup Language [Keating et al., 2020] is the standard serialisation format for constraint-based metabolic models built with tools such as COBRApy [Ebrahim et al., 2013]. SBML Level 2 and 3 files share a common XML namespace ending in `sbml`; the parser detects the format by matching the root element's local name.

Species elements map to `compound` nodes. Reaction elements map to `reaction` nodes. Stoichiometry comes from `<listOfReactants>` and `<listOfProducts>` children. Modifier species are classified by their SBO term [Courtot et al., 2011]: SBO:0000013 (catalyst) generates `CATALYZES` edges; SBO:0000020 (inhibitor) generates `INHIBITS` edges; other modifiers generate `ACTIVATES` edges.

A.3 BioPAX Parser

Biological Pathway Exchange Level 3 [Demir et al., 2010] is an OWL ontology serialised as RDF/XML, used by Reactome [Jassal et al., 2020], WikiPathways [Martens et al., 2021], and the NCI Pathway Interaction Database. Parsing requires the optional `rdflib` dependency, installed via the `biopax` extra. The parser performs SPARQL-style pattern matching over the RDF graph to extract:

- `SmallMolecule` instances → `compound` nodes.
- `Protein` instances → `enzyme` nodes.
- `BiochemicalReaction` instances → `reaction` nodes, with `left/right` properties becoming substrate and product edges, and `controller` properties becoming `CATALYZES` edges.
- `Pathway` instances → `pathway` nodes, with `memberPathwayComponent` links becoming `CONTAINS` edges.

A.4 CSV/TSV Parser

For custom or unpublished data, MetaKG accepts flat tables with a configurable column schema. The default column layout is:

`reaction_id`, `reaction_name`, `substrate`, `product`, `enzyme`, `stoich_substrate`, `stoich_product`, `pathway`, `ec_number`, `substrate_formula`, `enzyme_uniprot`

Multiple rows with the same `reaction_id` merge into a single reaction node—the standard way to encode multi-substrate or multi-product reactions in tabular form. A `CSVParserConfig` dataclass allows remapping all column names, making the parser suitable for lab-produced spreadsheets and bulk downloads from custom databases.

References

Anthropic. Claude: AI assistant. https://claude.ai, 2024a. Accessed February 2026.

Anthropic. Model Context Protocol (MCP): an open protocol that enables seamless integration between LLM applications and external data sources and tools. https://modelcontextprotocol.io, 2024b. Accessed February 2026.

Olivier Bodenreider. The Unified Medical Language System (UMLS): integrating biomedical terminology. *Nucleic Acids Research*, 32(suppl_1):D267–D270, 2004. doi: 10.1093/nar/gkh061.

Mélanie Courtot, Nick Juty, Christian Knüpfer, et al. Controlled vocabularies and semantics in systems biology. *Molecular Systems Biology*, 7:543, 2011. doi: 10.1038/msb.2011.77.

Emek Demir, Michael P. Cary, Suzanne Paley, et al. The BioPAX community standard for pathway data sharing. *Nature Biotechnology*, 28(9):935–942, 2010. doi: 10.1038/nbt.1666.

Ali Ebrahim, Joshua A. Lerman, Bernhard Ø. Palsson, and Daniel R. Hyduke. COBRApy: COnstraints-Based Reconstruction and Analysis for Python. *BMC Systems Biology*, 7:74, 2013. doi: 10.1186/1752-0509-7-74.

Xiyang Feng, Guodong Jin, Ziyi Chen, Chang Liu, and Semih Salihoğlu. KÙZU Graph Database Management System. In *Proceedings of the VLDB Endowment*, volume 16, pages 3630–3637, 2023. doi: 10.14778/3611540.3611556.

Flux Frontiers Contributors. CodeKG: A Structural and Semantic Knowledge Graph for Python Codebases. https://github.com/Flux-Frontiers/code_kg, 2025. Accessed February 2026.

Bijay Jassal, Lisa Matthews, Guilherme Viteri, et al. The reactome pathway knowledgebase. *Nucleic Acids Research*, 48(D1):D498–D503, 2020. doi: 10.1093/nar/gkz1031.

Minoru Kanehisa, Miho Furumichi, Yoko Sato, Mari Ishiguro-Watanabe, and Masumi Tanabe. KEGG: integrating viruses and cellular organisms. *Nucleic Acids Research*, 49(D1):D545–D551, 2021. doi: 10.1093/nar/gkaa970.

Sarah M. Keating, Dagmar Waltemath, Matthias König, et al. SBML Level 3: an extensible format for the exchange and reuse of biological models. *Molecular Systems Biology*, 16(8):e9110, 2020. doi: 10.15252/msb.20199110.

LanceDB Contributors. LanceDB: Developer-friendly, serverless vector database. https://lancedb.github.io/lancedb/, 2023. Accessed February 2026.

Marvin Martens, Ammar Ammar, Anders Riutta, et al. WikiPathways: connecting communities. *Nucleic Acids Research*, 49(D1):D613–D621, 2021. doi: 10.1093/nar/gkaa1024.

Sébastien Moretti, Kristian Paskov, Richard H. R. Hahnloser, et al. MetaNetX/MNXref: unified namespace for metabolites and biochemical reactions in the context of metabolic models. *Nucleic Acids Research*, 49 (D1):D570–D574, 2021. doi: 10.1093/nar/gkaa992.

Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, pages 3982–3992. Association for Computational Linguistics, 2019. doi: 10.18653/v1/D19-1410.

Ian Robinson, Jim Webber, and Emil Eifrem. *Graph Databases: New Opportunities for Connected Data*. O'Reilly Media, 2nd edition, 2015. ISBN 978-1-491-93089-2.

Paul Shannon, Andrew Markiel, Owen Ozier, et al. Cytoscape: a software environment for integrated models of biomolecular interaction networks. *Genome Research*, 13(11):2498–2504, 2003. doi: 10.1101/gr.1239303.

Streamlit Inc. Streamlit: The fastest way to build data apps. https://streamlit.io, 2019. Accessed February 2026.

C. Bane Sullivan and Alexander A. Kaszynski. PyVista: 3D plotting and mesh analysis through a streamlined interface for the Visualization Toolkit (VTK). *Journal of Open Source Software*, 4(37):1450, 2019. doi: 10.21105/joss.01450.

Helmut Vogel. A better way to construct the sunflower head. *Mathematical Biosciences*, 44(3-4):179–189, 1979. doi: 10.1016/0025-5564(79)90080-4.

Jason West. Pyvis: visualize and construct interactive network graphs in Python. https://pyvis.readthedocs.io, 2021. Accessed February 2026.